BrainAI

ProSuite Industrial

PROSUITE INDUSTRIAL

PRODUKT-DOKUMENTATION

BrainAI (Inhaber: Sascha Alexander Köhne)

BrainAI Inhaber: Sascha Alexander Köhne	PROSUITE INDUSTRIAL	🟢 ÖFFENTLICHE KLASSIFIZIERUNG
Core Architecture & Development	**Technical Documentation**	Zugriffsstufe: Öffentlich / Beta

---

Revisionshistorie / Revision History

Version	Datum	Autor	Status	Beschreibung
1.0.0	01.03.2026	S. Köhne	Stable	Initial Release: ProKM Kernel Architecture (Win/Linux)

1 Produktübersicht: ProKM (Process Kernel Module)

ProKM ist das tiefgreifendste Sicherheitsmodul der ProSuite Industrial. Während herkömmliche Kryptografie-Lösungen im User-Space (Ring-3) operieren und somit anfällig für Memory-Dumps, Side-Channel-Attacken und privilegierte Malware sind, verlagert ProKM die gesamte kryptografische Verarbeitung (ProEDC, ProHash, ProKey) direkt in den **Betriebssystem-Kernel (Ring-0)**.

Kernmerkmale der Architektur:

- **Isolierte Ausführung:** Schlüsselmaterial und Klartextdaten verlassen niemals den Kernel-Space.
- **Plattformübergreifende API:** Einheitliche IOCTL-Schnittstellen für Windows (KMDF) und Linux (Char Device).
- **Zero-Trust Memory Management:** Garantierte, physikalische RAM-Löschung vor der Speicherfreigabe.

2 System-Limits & Speichermanagement

Da Operationen im Kernel-Space kritische Auswirkungen auf die Systemstabilität (Kernel Panics / BSODs) haben können, implementiert ProKM strikte Sicherheitsgrenzen.

2.1 Das 32-Megabyte Transaktions-Limit

Um Speicherfragmentierung und Paging-Probleme im Kernel zu verhindern, ist jede einzelne IOCTL-Transaktion (Ver-/Entschlüsselung oder Hashing) auf exakt **32 Megabyte (`PROKM_MAX_ALLOC`)** limitiert.

- **Große Dateien:** Sollen Dateien > 32 MB verarbeitet werden, muss die User-Space-Applikation (z.B. der C# Wrapper) die Daten in Chunks von maximal 32 MB aufteilen und sequenziell an den Treiber senden.

2.2 Memory Allokation & Secure Wipe

- **Windows:** Der Treiber nutzt `ExAllocatePoolWithTag(NonPagedPoolNx, ...)`. Das bedeutet, der allozierte Speicher ist vor Ausführung geschützt (No-Execute) und kann nicht in die Auslagerungsdatei (Pagefile) geschrieben

werden. Nach der Operation wird der Speicher mit `RtlSecureZeroMemory` restlos überschrieben.

- **Linux:** Es wird `vmalloc()` genutzt, um große, zusammenhängende virtuelle Speicherblöcke sicher bereitzustellen. Die Bereinigung erfolgt über `memzero_explicit()`, um zu verhindern, dass der Compiler den Löschvorgang wegoptimiert.

3 IOCTL-Schnittstellen (Driver Calls)

Die Kommunikation zwischen der User-Space-Applikation und dem ProKM-Treiber erfolgt asynchron über standardisierte IOCTL-Calls. Die Magic-Number des Treibers ist `'k'`.

3.1 Unterstützte Befehle (Command Codes)

Befehl	Windows (CTL_CODE)	Linux (_IOWR)	Beschreibung
`IOCTL_PROKM_PROED`	`0x801`	`1`	Führt eine Ver- oder Entschlüsselung (AES/ProEDC) aus.
`IOCTL_PROKM_PROHASH`	`0x802`	`2`	Berechnet den 256-Bit Integritäts-Hash.
`IOCTL_PROKM_PROKEY`	`0x803`	`3`	Fordert Hardware-Entropie (Zufall) an.
`IOCTL_PROKM_STATUS`	`0x899`	`99`	Ruft den Health-Status des Treibers ab.

3.2 Datenstrukturen (ABI-Safe)

Die Speicherstrukturen sind mit `#pragma pack(push, 1)` verpackt, um plattformübergreifend Padding-Probleme zwischen 32-Bit und 64-Bit Systemen zu vermeiden.

1. ProED Encryption Request (`ProKM_ProED_Req`) Diese Struktur wird für `IOCTL_PROKM_PROED` verwendet. Der Puffer muss groß genug sein, um den Header und den variablen `payload` zu fassen.

```c
typedef struct {
    uint8_t  key[32];      // 256-Bit Master Key
    uint8_t  iv[16];       // 128-Bit Initialization Vector
    uint32_t mode;         // 0 = Encrypt, 1 = Decrypt
    uint32_t data_len;     // Länge des Payloads (Max 32 MB)
    uint8_t  payload[1];   // Start des variablen Datenstroms
} ProKM_ProED_Req;
```

2. ProHash Integrity Request (`ProKM_ProHash_Req`) Wird für `IOCTL_PROKM_PROHASH` verwendet. Der Kernel gibt die originalen Nutzdaten *und* den 32-Byte Hash am Ende des Payloads zurück.

```c
typedef struct {
    uint32_t data_len;     // Länge der zu hashenden Daten
    uint8_t  payload[1];   // Datenstrom
} ProKM_ProHash_Req;
```

4 Integration auf Betriebssystem-Ebene

4.1 Windows (KMDF)

Unter Windows wird der Treiber als `ProKM_Driver.sys` geladen.

- **Device Pfad:** Applikationen (z.B. in C#) müssen ein Datei-Handle auf `\\.\ProKM` (`\DosDevices\ProKM`) öffnen, um mittels `DeviceIoControl()` mit dem Kernel zu kommunizieren.
- **Sicherheit:** Der Treiber verifiziert sich beim Laden über den internen `ProKM_RunSelfTest()`. Schlägt dieser fehl (POST FAILED), verweigert der Treiber den Start (`STATUS_DEVICE_CONFIGURATION_ERROR`).

4.2 Linux (Character Device)

Unter Linux wird der Treiber als ladbares Kernel-Modul (`prokm.ko`) installiert.

- **Device Pfad:** Ein Node unter `/dev/prokm` wird erstellt. User-Space Programme öffnen diesen Node mit `open()` und senden Befehle via `ioctl()`.
- **Speicherschutz:** Der Treiber nutzt standardmäßig `copy_from_user` und `copy_to_user`, um Speicherzugriffsverletzungen (Segfaults) zu verhindern, falls fehlerhafte Pointer aus dem User-Space übergeben werden.

5 Fehlercodes & Troubleshooting

Wenn ein IOCTL-Call fehlschlägt, liefert der Treiber systemspezifische Fehlercodes zurück:

- **Windows `STATUS_INVALID_PARAMETER` / Linux `-EINVAL`:** Ausgelöst, wenn `data_len == 0` ist oder das hart einkodierte Limit von 32 MB (`PROKM_MAX_ALLOC`) überschritten wurde.
- **Windows `STATUS_BUFFER_TOO_SMALL` / Linux `-EFAULT`:** Der übergebene User-Space-Puffer ist kleiner als in `data_len` angegeben oder nicht groß genug, um die Antwortdaten aufzunehmen.
- **Windows `STATUS_INSUFFICIENT_RESOURCES` / Linux `-ENOMEM`:** Kritischer Systemzustand. Dem Kernel steht nicht genügend freier Speicher für die Allokation (`NonPagedPoolNx` / `vmalloc`) zur Verfügung.

© 2026 Sascha Köhne	✉️ **Contact:** koehne83@googlemail.com	🛡️ **ProTU-Verified**
System Architect	BrainAI Inhaber : Sascha Alexander Köhne	We don't need **BRUTEFORCE**, we know **PHYSICS**